

Lernziele:

- Funktionen mit Parametern, Rückgabewert und lokalen Variablen

Verwendete Architektur in dieser Session

In dieser Übung verwenden wir die Architektur, die in Session 10 verwendet wurde. Zur Erinnerung: In Session 10 wurde die ISA `ulm-ice40.isa` erweitert, um Funktionsaufrufe zu vereinfachen. Mit dem Kommando

```
ulm-generator --fetch ulm-ice40.isa
```

wurde die bisherige ISA-Beschreibung `ulm-ice40.isa` ins aktuelle Verzeichnis geladen. Dann wurde der Befehlssatz um zwei Befehle erweitert, indem folgende Zeilen hinzugefügt wurden:

```
0x0E    RR
:  call    %x
        ulm_absJump(ulm_regVal(x), x);

0x0F    U20_R
:  call    imm, %dest
        ulm_absJump(imm, dest);
```

Eine Mikroarchitektur in Form einer virtuellen Maschine und ein Assembler für diese ISA wurden dann mit

```
ulm-generator --install ulm-ice40.isa
```

gebaut.

Review von Session 18: Pattern mit Stackpointer und Framepointer

In Session 18 wurde am Ende für Funktionen ein Pattern verwendet, bei dem ein Stackpointer und ein Framepointer verwendet wurden. Die Funktionsweise wurde dabei nicht erklärt, sondern nur die Vorteile des Patterns experimentell bestätigt, die etwas mit der Buchhaltung zu tun hatten:

- Will eine Funktion auf ihre eigenen Parameter oder den Rückgabewert zugreifen, dann wird dafür dieser ominöse Framepointer verwendet. Zum Beispiel wird mit:

```
movq param0(%FP), %3
```

der erste Parameter in Register %3 geladen und mit

```
movq %3, retval(%FP)
```

der Inhalt von Register %3 als Rückgabewert hinterlegt.

- Will eine Funktion eine andere Funktion aufrufen, dann wird für die Übergabe von Parametern und für das Abholen des Rückgabewertes der Stackpointer verwendet. Zum Beispiel wird mit

```
movq %3, param0(%SP)
```

der Inhalt von Register %3 als erstes Argument übergeben und mit:

```
movq retval(%SP), %3
```

der erhaltene Rückgabewert in Register %3 geladen.

Man muss sich also nur diese Offsets (`retval`, `param0`, `param1`, etc.) merken, die man einmal festlegt, und wissen, in welcher Rolle man sich befindet:

- Will man zum Beispiel auf die eigenen Parameter zugreifen, dann ist man in der Callee-Rolle ("Ich wurde aufgerufen, was sind meine Parameter?").
- Will man Argumente übergeben, dann ist man in der Caller-Rolle ("Welche Argumente soll die Funktion als Parameter erhalten?").

Dieses Pattern kann man stupide nutzen, ohne zu wissen, wieso es funktioniert. Dass es funktioniert, wurde im Prologue und Epilogue versteckt, und diese kann man für jede Funktion stupide mit Copy & Paste jeweils am Anfang und Ende einfügen.

Aber obwohl es unnötig ist zu wissen, wie der Prologue und Epilogue funktioniert, macht es trotzdem mehr Spaß, ein bisschen Ahnung davon zu haben. Dazu kann man zunächst festhalten, welche Aufgaben der Prologue und Epilogue haben und wie beide zusammenspielen:

- Im Prologue wird die Rücksprungadresse auf den Stack gelegt. Im Epilogue wird die Rücksprungadresse vom Stack geladen und am Ende zur Rücksprungadresse gesprungen.
- Zusätzlich soll der Epilogue das Rückgängig machen was der Prologue getan hat. Wenn der Prologue zum Beispiel den Stackpointer ändert, dann muss der Epilogue den wieder auf den alten Wert zurücksetzen. Das gleiche gilt für nun eben auch für den Framepointer.

Prologue

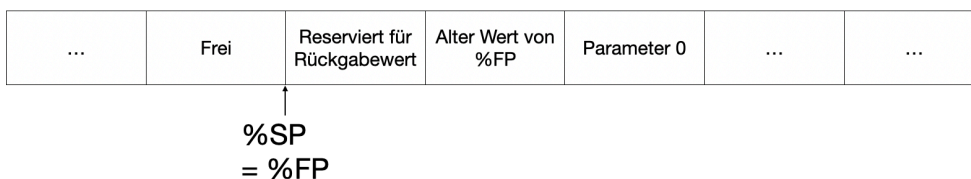
Im Prologue wird mit der ersten Instruktion die Rücksprungadresse auf den Stack gelegt. Mit der dritten Instruktion wird der Framepointer mit dem Wert des Stackpointers überschrieben. Damit dieses Überschreiben des Framepointers rückgängig gemacht werden kann, wird in der Instruktion davor (also mit der zweiten Instruktion) auch dieser ominöse Framepointer auf den Stack gelegt:

```

movq    %3,          (%SP)    // save return address
movq    %FP,         8(%SP)   // save old value of %FP
movq    %SP,         %FP      // overwrite %FP with %SP
// begin function body

```

Nach dem Prologue kann man den Stack (und diese zwei Pointer) also immer so beschreiben:



Epilogue

Was der Epilogue machen soll, ist klar, und die Details dazu kann man (wenn man will) später anschauen.

Aber was ist jetzt der Vorteil?

Der Vorteil ist, dass man nun abhängig von der Rolle Caller oder Callee bequem auf die richtigen Speicherflächen zugreifen kann. Und das sieht man, wenn man Beispiele für Funktionsaufrufe betrachtet.

Nehmen wir ein blödes Beispiel, bei dem eine Funktion `bar` den eigenen Parameter `a` als Argument an eine Funktion `foo` übergibt und den Rückgabewert von `foo` als eigenen Rückgabewert übernimmt:

```

1 fn foo(a: int): int
2 {
3     // ...
4 }
5
6 fn bar(a: int): int
7 {
8     return foo(a);
9 }

```

In Zeile 8 wird also `bar` zum Caller, muss aber gleichzeitig auch auf den eigenen Parameter zugreifen, ist also auch gleichzeitig ein Callee. Das hört sich kompliziert an und ist es auch. Und hier

hilft das Pattern, wenn man sich anschaut, wie man Zeile 8 damit umsetzt:

```

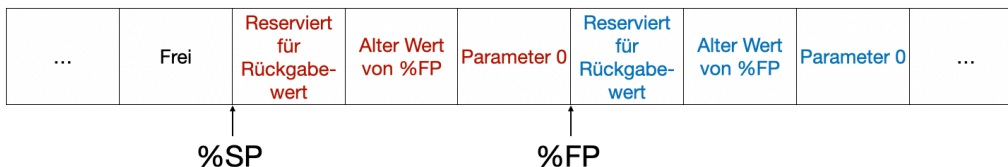
1  subq    8*(3 + 1),    %SP,    %SP // space for one argument
2  movq    param0(%FP),  %3      // load first parameter
3  movq    %3,           param0(%SP) // .. and pass as first argument
4  call    foo,          %3
5  movq    retval(%SP),  %3      // load returned value
6  movq    %3,           retval(%FP) // .. and use it as own return value
7  addq    8*(3 + 1),    %SP,    %SP

```

Nach der Instruktion in Zeile 1 kann man den Stack so beschreiben:

Zeug für foo

Zeug von bar



Ich will jetzt bestimmt nicht behaupten, dass damit die restlichen Instruktionen trivial sind. Aber es ist jetzt möglich, dass ein Mensch die Zeilen versteht und selbst programmieren kann. Zum Beispiel wird mit

```
2  movq    param0(%FP),  %3      // load first parameter
```

der eigene Parameter (in der Callee-Rolle) in Register `%3` geladen und mit

```
3  movq    %3,           param0(%SP) // .. and pass as first argument
```

der Inhalt von Register `%3` als Argument (in der Caller-Rolle) übergeben. Analog wird nach dem Funktionsaufruf (in der Caller-Rolle) der Rückgabewert von `foo` mit:

```
5  movq    retval(%SP),  %3      // load returned value
```

in Register `%3` geladen und mit:

```
6  movq    %3,           retval(%FP) // .. and use it as own return value
```

als eigener Rückgabewert hinterlegt (in der Callee-Rolle).

Schritt 1 Ausgabe einer nicht-negativen ganzen Zahl

Als Motivation dafür, dass das Pattern noch weiterentwickelt werden muss, um auch lokale Variablen auf dem Stack speichern zu können, soll folgendes ABC-Programm dienen. Es implementiert eine Funktion `printUnsigned`, um ein Unsigned-Integer auszugeben. Für die Ausgabe wird nur die Funktion `putchar` verwendet. Damit kann dieses Programm später mit dem erweiterten Pattern relativ einfach in Assembler umgeschrieben werden:

```

1 @ <stdio.h>
2
3 fn printUnsigned(a: u64)
4 {
5     local r: u64 = a % 10;
6     local q: u64 = a / 10;
7     if (q) {
8         printUnsigned(q);
9     }
10    putchar('0' + r);
11 }
12
13 fn main()
14 {
15     printUnsigned(1234);
16     putchar('\n');
17 }

```

Aufgabe: Testet das Programm und erklärt eurem Nachbarn die Idee hinter der Implementierung von `printUnsigned`. Insbesondere sollte euer Nachbar dann verstehen, warum der rekursive Aufruf in Zeile 8 vor der Ausgabe in Zeile 10 erfolgen muss.

Schritt 2 Hardware pimpen

Die ulm-ice40 Architektur unterstützt nur ALU-Instruktionen, die sich auf dem iCE40-FPGA so umsetzen lassen, dass diese in einem Takt ausgeführt werden können. Zum Beispiel eine 16-Bit-Multiplikation, aber keine 64-Bit-Multiplikation. Bei der Division sieht es noch düsterer aus, dafür gibt es gar keine Instruktion. Will man auf der ulm-ice40 Architektur eine 64-Bit-Multiplikation und eine 64-Bit-Division, dann muss man dies softwareseitig umsetzen, indem man dies auf die verfügbaren Instruktionen zurückführt. Dazu kann man Algorithmen verwenden, die analog zum schriftlichen Multiplizieren und Dividieren arbeiten und unter anderem Bit-Shift-Operationen verwenden.

Um hier aber nicht den Fokus zu verlieren, gehen wir den bequemen Weg und erweitern die Architektur so, dass die Operationen, die wir benötigen, direkt unterstützt werden. Wie man das dann in Hardware umsetzt, ist ein anderes Problem. Ergänzt die Datei ulm-ice40.isa um folgende Zeilen:

```
0x40 U16_R_R
:   idivq imm, %y, %z
    ulm_div128(imm, ulm_regVal(y), ulm_regVal(0), z, 0, 0);

0x41 U16_R_R
:   imodq imm, %y, %z
    ulm_div128(imm, ulm_regVal(y), ulm_regVal(0), 0, 0, z);

0x42 R_R_R
:   imulq %x, %y, %z
    ulm_mul64(ulm_regVal(x), ulm_regVal(y), z);
```

Damit habt ihr drei neue Assembler-Instruktionen, die hier anhand von Beispielen erklärt werden:

- `idivq 10, %3, %4` teilt den Inhalt von Register `%3` durch `10` und schreibt den ganzzahligen Quotienten in Register `%4`.
- `imodq 10, %3, %4` teilt den Inhalt von Register `%3` durch `10` und schreibt den Rest der Division in Register `%4`.
- `imulq %3, %4, %5` multipliziert Register `%3` mit Register `%4`. Das Ergebnis wird auf 64 Bit abgeschnitten und in Register `%5` geschrieben.

Schritt 3 Lokale Variablen

Bisher haben wir für lokale Variablen einfach Register verwendet. Der Vorteil ist, dass dies nicht nur sehr effizient, sondern auch einfach zu programmieren ist. Der Nachteil ist, dass dies im Allgemeinen nur in Leaf-Funktionen funktioniert. Denn wenn eine andere Funktion aufgerufen wird, kann diese unsere Register überschreiben. Im Allgemeinen kann man für lokale Variablen deshalb den Stack verwenden. Im Pattern muss man dazu nur den Prologue um eine Zeile erweitern und zusätzliche Equ-Direktiven für Offsets verwenden. Den Epilogue muss man nicht ändern.

Das Ganze soll am Beispiel des obigen ABC-Programms mit zwei lokalen Variablen erklärt werden. Den Prologue erweitert man zu:

```
movq    %3,          (%SP)    // save return address
movq    %FP,         8(%SP)   // save old value of %FP
movq    %SP,         %FP     // overwrite %FP with %SP
subq    2 * 8,       %SP,     %SP # two local variables
// begin function body
```

Durch die zusätzliche `subq`-Instruktion sieht der Stack nach dem Prologue so aus:



Um bezüglich des Framepointers auf die Parameter und den Rückgabewert zugreifen zu können, kann man die bisherigen Offsets (`retval`, `param0`, etc.) verwenden. Auf dem Stack ist jetzt aber noch zusätzlich Platz für zwei lokale Variablen reserviert (mit einer Größe von jeweils 8 Bytes). Für die Offsets kann man dazu die Symbole `local0` und `local1` mit:

```
.equ local0, -8
.equ local1, local0 - 8
```

vereinbaren. Und diese Liste kann man natürlich nach Belieben erweitern. Nun kann man den Funktionskörper nach einem einfachen Schema Zeile für Zeile in Assembler übersetzen.

Zum Beispiel kann man

```
5      local r: u64 = a % 10;
```

umschreiben in

```
movq param0(%FP), %3
imodq 10, %3, %3
movq %3, local0(%FP)
```

Man muss sich dann nur merken, dass `param0` das Offset für den Parameter `a` und `local0` das Offset für die lokale Variable `r` ist. Man kann es natürlich noch hübscher machen, wenn man nach dem Prologue noch spezielle Equ-Direktiven vereinbart. Zum Beispiel:

```
.equ a, param0
.equ r, local0
.equ q, local1
```

Dann kann man schreiben:

```
movq a(%FP), %3
imodq 10, %3, %3
movq %3, r(%FP)
```

Aufgabe: Schreibt das Programm für die Ausgabe eines Unsigned-Integer in Assembler.

Schritt 4

Jetzt könnt ihr euch natürlich austoben. Wir haben nun Assembler-Funktionen für das Einlesen eines Unsigned-Integer, für die Ausgabe eines Strings und für die Ausgabe eines Unsigned-Integer. Man könnte jetzt ein Programm schreiben, das den Benutzer zunächst bittet, eine Zahl `n` einzugeben. Es berechnet dann die Fakultät von `n` und gibt das Ergebnis aus. Also etwas, das dann in der Benutzung so aussieht:

```
MCL$ ./a.out
n = 5
5! = 120
```

Oder man macht sich das Leben einfach und schreibt einen Compiler, der das Pattern verwendet und uns den Assembler-Code generiert.

Mögliche Implementierung für Schritt 3

```

.equ    SP,      1
.equ    FP,      2

.equ    retval,  16
.equ    param0,  24
.equ    param1,  param0 + 8
.equ    param2,  param1 + 8

.equ    local0,  -8
.equ    local1,  local0 - 8

loadz   0,          %SP

# call: int main(void)
subq    8*(3 + 0),   %SP,      %SP
call    main,        %3
movq    retval(%SP), %3
addq    8*(3 + 0),   %SP,      %SP
halt    %3

main:
movq    %3,          (%SP)
movq    %FP,          8(%SP)
movq    %SP,          %FP
// begin function body

subq    8*(3 + 1),   %SP,      %SP
loadz   1234,        %3
movq    %3,          param0(%SP)
call    printUnsigned, %3
addq    8*(3 + 1),   %SP,      %SP

putc    '\n'

// end function body
movq    %FP,          %SP
movq    8(%SP),       %FP
movq    (%SP),        %3
ret     %3

printUnsigned:
movq    %3,          (%SP)
movq    %FP,          8(%SP)
movq    %SP,          %FP
subq    2 * 8,        %SP,      %SP
// begin function body

.equ    a,          param0
.equ    r,          local0
.equ    q,          local1

movq    a(%FP),       %3
imodq   10,           %3,      %3
movq    %3,          r(%FP)

movq    a(%FP),       %3
idivq   10,           %3,      %3
movq    %3,          q(%FP)

movq    q(%FP),       %3
subq    0,            %3,      %0
jz      .endif

subq    8*(3 + 1),   %SP,      %SP
movq    q(%FP),      %3
movq    %3,          q(%SP)
call    printUnsigned, %3
addq    8*(3 + 1),   %SP,      %SP

.endif:
movq    r(%FP),       %3
addq    '0',          %3,      %3
putc    %3

// end function body
movq    %FP,          %SP
movq    8(%SP),       %FP
movq    (%SP),        %3
ret     %3

```